

INSTITUCIÓN UNIVERSITARIA DE COLOMBIA

SESIÓN 1 — INTRODUCCIÓN A SISTEMAS EXPERTOS Y REPASO PYTHON

I. DATOS GENERALES DE LA ASIGNATURA

Asignatura:	Sistemas Expertos e Inteligencia Artificial
Período Académico:	VIII Semestre
Créditos:	3 créditos
Duración en Horas:	40 horas
Docente:	Amaury Giovanni Méndez Aguirre

CONTEXTO: CAPTURANDO LA EXPERIENCIA HUMANA

A diferencia de las Redes Neuronales que aprenden patrones estadísticos a partir de grandes volúmenes de datos (enfoque conexionista), los **Sistemas Expertos (SE)** pertenecen a la rama de la Inteligencia Artificial Simbólica. Su objetivo es emular la capacidad de toma de decisiones de un experto humano en un dominio muy específico (por ejemplo, diagnóstico médico o auditoría financiera).

La premisa fundamental es aislar el conocimiento puro (las reglas del negocio) del algoritmo que las procesa. Esto permite que el sistema sea auditable: cuando un SE toma una decisión financiera, puede explicar exactamente qué pasos lógicos siguió para llegar a esa conclusión, algo indispensable en entornos corporativos legales.

1. Arquitectura Fundamental de un Sistema Experto

Todo Sistema Experto clásico se compone de tres módulos estrictamente separados:

- **Base de Conocimientos (Knowledge Base):** Contiene el conocimiento del dominio a largo plazo. Se divide en dos:
 - *Base de Reglas:* Estructuras lógicas de la forma SI [premisa] ENTONCES [conclusión].
 - *Base de Hechos (Memoria de Trabajo):* La información específica del problema actual que se está resolviendo.
- **Motor de Inferencia (Inference Engine):** El "cerebro" algorítmico. Es un programa que cruza los Hechos actuales con las Reglas para deducir nuevos hechos.
- **Interfaz de Usuario / Módulo de Explicación:** Interactúa con el usuario para pedir datos faltantes y justificar la decisión final.

TALLER ANALÍTICO 1: LÓGICA PROPOSICIONAL (20 MIN)

En ingeniería del conocimiento, debemos traducir el lenguaje natural a reglas lógicas estrictas. Lea el siguiente fragmento de un manual de otorgamiento de créditos:

"Un cliente es elegible para el crédito Hipotecario Plus si sus ingresos anuales superan los 50,000 USD y tiene un historial crediticio 'Excelente'. Sin embargo, si el cliente tiene deudas activas superiores a 10,000 USD, no será elegible bajo ninguna circunstancia, a menos que presente un avalista."

Misión: Extraiga y formalice este texto en Reglas de Producción (SI... Y/O... ENTONCES...). Identifique claramente cuáles son las variables (Hechos) que el sistema debe pedir al usuario.

Escriba sus reglas lógicas aquí...

2. Implementación en Python: Diccionarios y Lógica de Primer Orden

Para construir la Memoria de Trabajo en Python, la estructura de datos más eficiente es el **Diccionario (Hash Map)**. Permite almacenar los Hechos como pares Clave-Valor, otorgando un acceso computacional de tiempo constante $O(1)$.

Las Reglas se pueden modelar inicialmente como funciones puras que evalúan la memoria de trabajo y retornan un nuevo estado.

```
# 1. MEMORIA DE TRABAJO (Base de Hechos)
estado_cliente = {
    "ingresos": 60000,
    "historial": "Excelente",
    "deuda_activa": 12000,
    "tiene_avalista": True
}

# 2. BASE DE REGLAS Y MOTOR DE INFERENCIA ESTÁTICO
def motor_evaluacion_credito(hechos):
    # Regla de Rechazo Absoluto (Precedencia alta)
    if hechos["deuda_activa"] > 10000 and not hechos["tiene_avalista"]:
        return "RECHAZADO: Alto riesgo por deuda sin aval."

    # Regla de Aprobación
    if hechos["ingresos"] > 50000 and hechos["historial"] == "Excelente":
        return "APROBADO: Cumple criterios Hipotecario Plus."

    # Regla por defecto (Fallback)
    return "EN REVISIÓN MANUAL: No cumple criterios automáticos."

# Ejecución
decision = motor_evaluacion_credito(estado_cliente)
print("Veredicto:", decision)
```

TALLER DE LABORATORIO: SISTEMA DE DIAGNÓSTICO IT (40 MIN)

Reto en Python: Van a programar el prototipo de un Sistema Experto para el HelpDesk de una empresa tecnológica.

1. Creen un diccionario llamado `servidor_estado` con al menos 4 métricas técnicas (ej. "cpu_uso", "memoria_libre", "ping_respuesta", "temperatura").
2. Programen una función `diagnosticar_servidor(hechos)` que contenga al menos 3 reglas lógicas anidadas (usando `if`, `elif` y operadores lógicos `and/or`).
3. Una de las reglas debe identificar un estado crítico (ej. `temperatura > 80` Y ventilador apagado) y otra un estado de advertencia.
4. Impriman el diagnóstico. Cambien los valores del diccionario original para probar que el motor de inferencia toma diferentes caminos (cobertura de ramas).